

Extracting Higher-Order Goals from the Mizar Mathematical Library

Chad E. Brown and Josef Urban^(✉)

Czech Technical University, Prague, Czech Republic
Josef.Urban@gmail.com

Abstract. Certain constructs allowed in Mizar articles cannot be represented in first-order logic but can be represented in higher-order logic. We describe a way to obtain higher-order theorem proving problems from Mizar articles that make use of these constructs. In particular, higher-order logic is used to represent schemes, a global choice construct and set level binders. The higher-order automated theorem provers Satallax and LEO-II have been run on collections of these problems and the results are discussed.

Keywords: Formalized mathematics · Set theory · Higher-order logic · Automated theorem proving

1 Introduction

The Mizar Problems for Theorem Proving (MPTP) system has been developed and used to extract first-order theorem proving problems from the Mizar Mathematical Library (MML) [14–16]. However, some aspects of the Mizar language cannot be directly represented in first-order. In particular, Mizar provides supports for *Schemes* (allowing some degree of quantification over predicates and functions), *Fraenkel terms* (allowing sets to be specified using term level binders such as $\{f(x)|x \in A, p(x)\}$) and a *global choice operator* **the** on types [6]. In order to obtain first-order problems, the MPTP has dealt with schemes used in a proof by exporting the first-order instances of the scheme used in the proof. Additionally, Fraenkel terms and global choice have been made first-order by a process of deanonymization [14].

We describe an extension of MPTP targeting higher-order logic. Schemes can be represented directly in higher-order logic since quantifiers over predicates and functions are allowed. Instead of giving the instances of schemes used in a proof, schemes are exported as second-order formulas (relying on the problem solver to find appropriate instances). Global choice can be represented by a selection operator on the type of individuals and a corresponding choice axiom. We also give a method for representing Fraenkel terms, though these are more challenging both to represent and to reason about.

J. Urban—This work was supported by ERC Consolidator grant nr. 649043 *AI4REASON*.

The resulting system has been used to extract a collection of higher-order theorem proving problems in THF0 format [9]. As in [14] we can partition the problem set into simple justifications (the Mizar **by** steps – or sometimes no explicit justification), scheme justifications (the Mizar **from** steps indicating application of a scheme) and theorems (including schemes proven in the MML). There are roughly 10192 scheme justifications throughout Mizar proofs in the MML, and we consider the higher-order problems corresponding to all of them. For simple justifications, we focus only on those involving global choice or Fraenkel terms and restrict ourselves to such steps in only four Mizar articles, giving 245 higher-order problems involving Fraenkel terms and 47 problems involving the global choice operator. For theorems, we focus only on 610 proven schemes whose proof in the MML requires a scheme justification. We describe some examples and results from running the higher-order automated theorem provers Satallax [3] and LEO-II [2] on some collections of these problems.

In Sect. 2 we give a short description of the syntax of higher-order logic. In Sect. 3 we define M-types, M-terms and M-propositions corresponding to an idealized version of the Mizar language. In Sect. 4 we describe the mapping of M-types, M-terms and M-propositions into higher-order terms, with a focus on the higher-order aspects. Section 5 describes experiments using Satallax and LEO-II on the resulting higher-order problems.

2 Syntax of Higher-Order Logic

We give a short introduction to the syntax of higher-order logic (in the form of Church’s simple type theory [4]) so that we can describe the mapping in Sect. 4. In order to present higher-order problems to theorem provers, the THF0 format is used [9], but we mostly restrict ourselves to mathematical presentations of higher-order terms here.

There are two base types o (for propositions) and ι (for individuals, which will always be sets for us). The remaining types are function types $\alpha\beta$ where α and β are types. The type $\alpha\beta$ is the type of functions from α to β (and is sometimes written $\alpha \rightarrow \beta$).

We assume there are infinitely many variables x at each type α . We sometimes write the type as a subscript to make it clear, as in x_α . Likewise, there may be arbitrarily many constants c at each type α . We freely generate the set of typed terms as follows:

- A variable x of type α is a term of type α .
- A constant c of type α is a term of type α .
- If s is a term of type $\alpha\beta$ and t is a term of type α , then (st) is a term of type β .
- If x is a variable of type α and s is a term of type β , then $(\lambda x.s)$ is a term of type $\alpha\beta$.
- $\top$ is a term of type o .
- If s and t are terms of type α , then $(s =_\alpha t)$ is a term of type o .
- If s is a term of type o , then $(\neg s)$ is a term of type o .

- If s and t are terms of type o , then $(s \wedge t)$, $(s \vee t)$, $(s \rightarrow t)$ and $(s \leftrightarrow t)$ are terms of type o .
- If x is a variable of type α and s is a term of type o , then $(\forall x.s)$ and $(\exists x.s)$ are terms of type o .

Terms of type o are also called propositions.

We omit parentheses with the following conventions:

- Application associates to the left, e.g., stu means $((st)u)$.
- Binders have as large a scope as possible, e.g., both x are bound in $\forall x_o.x \vee \neg x$.
- The connectives $\rightarrow$, $\wedge$ and $\vee$ are considered right associative.
- The precedence of the binary and unary connectives are $=_\alpha$, $\neg$, $\wedge$, $\vee$, $\rightarrow$ and finally $\leftrightarrow$.

In addition, we omit the type subscript on $=$ when it is clear, and we write $s \neq t$ for $\neg(s = t)$. Likewise we may write several binders together, as in $\forall xyz_\alpha.s$ for $\forall x.\forall y.\forall z.s$ where x , y and z should all have type α .

3 Idealized Mizar

In order to describe the translation from Mizar to Higher-Order Logic we first give a short presentation of an idealized subset of the Mizar language. For a full presentation of the Mizar language, we direct the reader to [6].

To simplify the presentation, we assume that some variables and constants of higher-order logic are also variables and constants of Mizar, and that the translation will simply map variables and constants to themselves. The language of Mizar is restricted in a way that only variables and constants of certain types can be used:

- We call variables of type ι *object variables* and call constants of type ι *object constants*.
- For each $n \geq 1$, we call variables of simple type $\underbrace{\iota \dots \iota}_n$ *function variables (of arity n)*. Likewise, we call constants of this type *function constants (of arity n)*. We use F and G to range over function variables and f and g to range over function constants.
- For each $n \geq 0$, we call variables of simple type $\underbrace{\iota \dots \iota}_n o$ *predicate variables (of arity n)*. Likewise, we call constants of this type *predicate constants (of arity n)*. We use P and Q to range over predicate variables and p and q to range over predicate constants.

Mizar quantifiers only bind object variables. Predicate variables and function variables only appear in schemes and are listed (with typing information) in the prefix of a scheme.

Mizar articles typically consist of definitions and theorems (some of which are schemes). A definition may be of an object constant, a function constant

or a predicate constant. Predicate constants are sometimes defined as *modes* or *attributes*, which can then be used to construct Mizar types. Mizar types can be thought of as predicates over the universe of discourse. Mizar insists that types are nonempty and that all types, terms and propositions are well-typed (in Mizar's typing system).

In our idealized version of Mizar, we can ignore these restrictions and define more liberal sets of M-types, M-terms and M-propositions by mutual recursion. The intention is that Mizar types, terms and propositions (at least within the subset of Mizar considered in this article) will give M-types, M-terms and M-propositions, although not all M-types, M-terms and M-propositions would be accepted by Mizar.

M-types $A, B, \dots$ are generated as follows:

- **set** is an M-type.
- If p is an $n + 1$ -ary predicate constant and $T_1, \dots, T_n$ are M-terms, then $p(\cdot, T_1, \dots, T_n)$ is an M-type. (Here p is playing the role of a Mizar *mode*.)
- If q is a unary predicate constant and A is an M-type, then $q\ A$ and **non** $q\ A$ are M-types. (Here q is playing the role of a Mizar *attribute*.)

M-terms $S, T, \dots$ are generated as follows:

- An object variable x is an M-term.
- An object constant c is an M-term.
- If F is a function variable of arity n and $T_1, \dots, T_n$ are M-terms, then $F(T_1, \dots, T_n)$ is an M-term.
- If f is a function constant of arity n and $T_1, \dots, T_n$ are M-terms, then $f(T_1, \dots, T_n)$ is an M-term.
- If A is an M-type, then (**the** A) is an M-term. (The **the** is called a *global choice operator*.)
- If $x_1, \dots, x_n$ are object variables, $A_1, \dots, A_n$ are M-types, T is an M-term and Φ is an M-proposition, then $\{T\ \textbf{where}\ x_1\ \textbf{is}\ A_1, \dots, x_n\ \textbf{is}\ A_n : \Phi\}$ is an M-term. (These are called *Fraenkel terms*.)

M-propositions $\Phi, \Psi, \dots$ are generated as follows:

- If P is an n -ary predicate variable of arity n and $T_1, \dots, T_n$ are M-terms, then $P(T_1, \dots, T_n)$ is an M-proposition.
- If p is an n -ary predicate constant of arity n and $T_1, \dots, T_n$ are M-terms, then $p(T_1, \dots, T_n)$ is an M-proposition.
- If S and T are M-terms, then $(S = T)$ and $(S\ \textbf{in}\ T)$ are M-propositions.
- If Φ is an M-proposition, then (**not** Φ) is an M-proposition.
- If Φ and Ψ are M-propositions, then $(\Phi\ \&\ \Psi)$, $(\Phi\ \textbf{or}\ \Psi)$, $(\Phi\ \textbf{implies}\ \Psi)$ and $(\Phi\ \textbf{iff}\ \Psi)$ are M-propositions.
- If x is an object variable, A is an M-type and Φ is an M-proposition, then $(\textbf{for}\ x\ \textbf{being}\ A\ \textbf{holds}\ \Phi)$ and $(\textbf{ex}\ x\ \textbf{being}\ A\ \textbf{st}\ \Phi)$ are M-propositions.

Most Mizar theorems correspond to M-propositions. However, in some cases (namely, schemes) there are function variables or predicate variables which cannot be bound by quantifiers. We now define the notion of a prefix to list such

variables. When translating to higher-order propositions, the prefix will determine the outermost quantifiers.

A variable declaration is one of the following:

- $x : A$ where x is an object variable and A is an M-type.
- $F(A_1, \dots, A_n) : B$ where F is a function variable of arity n and $A_1, \dots, A_n, B$ are M-types.
- $P[A_1, \dots, A_n]$ where P is a predicate variable of arity n and $A_1, \dots, A_n$ are M-types.

A prefix is a list of variable declarations.

An M-statement (Γ, Φ) is a prefix Γ and an M-proposition Φ . For Mizar theorems other than schemes, the prefix Γ will always be empty. Some Mizar schemes will declare what appears to be a function variable of arity 0. In such a case, we use object variables instead. (This is why object variable declarations are allowed in a prefix.)

An example of a scheme is Separation: for each set A and predicate P , there is a set X such that $x \in X$ iff $x \in A$ and $P(x)$ [5].

```
scheme Separation { A()-> set, P[set] } :
  ex X being set st for x being set holds x in X iff x in A() & P[x]
```

The M-statement in this case is (Γ, Φ) where Γ is the prefix $A : \text{set}, P[\text{set}]$ (declaring an object variable A of type **set** and a predicate variable P of arity 1) and Φ is the M-proposition

ex X **being set st for** x **being set holds** x **in** X **iff** x **in** A **&** $P(x)$

corresponding to the body of the scheme.

4 Mapping Mizar to Higher-Order Logic

We now describe a mapping from M-types, M-terms, M-propositions and M-statements to higher-order terms, concentrating on the aspects that require higher-order constructs. The base type ι will correspond to Mizar objects (sets). We will use $\ulcorner - \urcorner$ to denote the image of an M-type, M-term, M-proposition or M-statement as a term in higher-order logic under the translation. The intention is that mapping $\ulcorner - \urcorner$ should send M-statements corresponding to Mizar theorems to provable propositions in higher-order logic. To be precise about this would require giving details about the proof theory of Mizar, which is beyond the scope of this paper.

In order to specify the translation, we need to declare a family of constants the higher-order problems may make use of. A special relation in Mizar is set membership (**in**), translated as **r2.hidden** by the MPTP system. For this reason, we include a declared constant **r2.hidden** of type ιo in the higher-order setting. For readability, we will write $s \in t$ for the term **r2.hidden** s t . (We will also write $s \notin t$ for $\neg(s \in t)$.) This allows us to translate an M-proposition S **in** T

simply as $\lceil S \rceil \in \lceil T \rceil$. We also declare a constant ε of type $(\iota o)\iota$. This allows use to translate an M-term (**the** A) as $\varepsilon \lceil A \rceil$. Finally, we need a family of constants for translating Fraenkel terms. For this purpose we declare a constant replSep_n of type

$$(\iota o)(\iota o) \cdots (\underbrace{\iota \cdots \iota o}_n)(\underbrace{\iota \cdots \iota \iota}_n)(\underbrace{\iota \cdots \iota o}_n)\iota$$

for each n . (In practice only a finite number of these can be declared in a single problem, and we declare them up to the maximum n required to translate the problem. When translating the MML the maximum required n was 6.) We can use replSep_n to translate $\{T \text{ where } x_1 \text{ is } A_1, \dots, x_n \text{ is } A_n : \Phi\}$ as

$$\text{replSep}_n \lceil A_1 \rceil (\lambda x_1. \lceil A_2 \rceil) \cdots (\lambda x_1 \cdots x_{n-1}. \lceil A_n \rceil) (\lambda x_1 \cdots x_n. \lceil T \rceil) (\lambda x_1 \cdots x_n. \lceil \Phi \rceil).$$

Before giving the translation, let us also remark on the intended semantics of these new constants. The constant ε is a choice operator so that εp satisfies p unless p is empty. The remaining constants are set theory related, and are required since the Mizar language targets set theory. In particular, the MML is based on Tarski-Grothendieck Set Theory (TG). For this reason, we take the intended interpretation of ι as a model of TG. The constant r2_hidden is intended to be membership on this model. The replSep_n constants give ways to specify sets. For simplicity, we consider only the $n = 1$ case. A first approximation would be to think of $\text{replSep}_1 s (\lambda x.t) (\lambda x.u)$ as a set $\{t \mid x \in s, u\}$. However, s has type ιo , not type ι , so we should write $\{t \mid x : sx \wedge u\}$. In general, if s is a predicate that corresponds to a class instead of a set, $\{t \mid x : sx \wedge u\}$ will not be a set. Mizar avoids this problem by enforcing an extra condition when Fraenkel terms are used: all the types $A_1, \dots, A_n$ must satisfy a “sethood” condition: that the collection of all elements of the type are contained in a bounding set. In the higher-order problems we define a corresponding constant **sethood** of type $(\iota o)o$ as follows:

$$\lambda p_{\iota o}. \exists y_{\iota}. \forall x_{\iota}. px \rightarrow x \in y.$$

Then we can interpret $\text{replSep}_1 s (\lambda x.t) (\lambda x.u)$ to be $\{t \mid x : sx \wedge u\}$ if **sethood** s holds and interpret $\text{replSep}_1 s (\lambda x.t) (\lambda x.u)$ to be the empty set otherwise. The new constants and corresponding axioms for the higher-order problems are given in Fig. 1. For each n there are two axioms for replSep_n : an introduction axiom replSepI_n and an elimination axiom replSepE_n . The sethood conditions are only required for replSepI_n since the intended interpretation of replSep_n is the empty set when applied to an argument for which the sethood condition is violated. In practice, **sethood** and replSep_n (for $n \geq 1$) are only included if the problem contains a Fraenkel term.

Each M-type A will map to a term $\lceil A \rceil$ of type ιo (a predicate or class), each M-term T will map to a term $\lceil T \rceil$ of type ι (a set) and each M-proposition Φ will map to a term $\lceil \Phi \rceil$ of type o (a proposition). Note that Mizar has dependent types and so an M-type A and the corresponding predicate $\lceil A \rceil$ may contain free variables. The mapping is defined by recursion as given in Fig. 2. Note that while we take $\lceil x \rceil = x$ and $\lceil c \rceil = c$ in principle, variables and constants are

$$\begin{aligned}
\varepsilon &: (\iota o) \iota & \mathbf{epsax} &: \forall p_{\iota o}. \forall x_{\iota}. px \rightarrow p(\varepsilon p) & \mathbf{r2_hidden} &: \iota o \\
\mathbf{sethood} &: (\iota o) o := \lambda p_{\iota o}. \exists y_{\iota}. \forall x_{\iota}. px \rightarrow x \in y & \mathbf{replSep}_1 &: (\iota o)(\iota o)(\iota o) \iota \\
\mathbf{replSepI}_1 &: \forall A_{\iota o}. \forall f_{\iota \iota}. \forall P_{\iota o}. \forall x_{\iota}. \mathbf{sethood} A \rightarrow Ax \rightarrow Px \rightarrow fx \in \mathbf{replSep}_1 A f P \\
\mathbf{replSepE}_1 &: \forall A_{\iota o}. \forall f_{\iota \iota}. \forall P_{\iota o}. \forall y_{\iota}. y \in \mathbf{replSep}_1 A f P \rightarrow \exists x_{\iota}. Ax \wedge Px \wedge y = fx \\
\mathbf{replSep}_2 &: (\iota o)(\iota o)(\iota o)(\iota o) \iota \\
\mathbf{replSepI}_2 &: \forall A_{\iota o}. \forall B_{\iota o}. \forall f_{\iota \iota}. \forall P_{\iota o}. \forall xy_{\iota}. \mathbf{sethood} A \rightarrow (\forall x_{\iota}. Ax \rightarrow \mathbf{sethood} (Bx)) \rightarrow \\
& Ax \rightarrow Bxy \rightarrow Pxy \rightarrow fxy \in (\mathbf{replSep}_2 A B f P) \\
& \dots \\
\mathbf{replSep}_n &: (\iota o)(\iota o) \dots (\underbrace{\iota \dots \iota}_n o) (\underbrace{\iota \dots \iota}_n \iota) (\underbrace{\iota \dots \iota}_n o) \iota \\
& \dots
\end{aligned}$$

Fig. 1. Higher-order declarations

$$\begin{aligned}
\lceil \mathbf{set} \rceil &= \lambda x. \top & \lceil p(\cdot, T_1, \dots, T_n) \rceil &= \lambda x. p \ x \ \lceil T_1 \rceil \ \dots \ \lceil T_n \rceil^* \\
\lceil q \ A \rceil &= \lambda x. q \ x \wedge \lceil A \rceil^* & \lceil \mathbf{non} \ q \ A \rceil &= \lambda x. \neg q \ x \wedge \lceil A \rceil^* & \lceil x \rceil &= x & \lceil c \rceil &= c \\
\lceil F(T_1, \dots, T_n) \rceil &= F \ \lceil T_1 \rceil \ \dots \ \lceil T_n \rceil & \lceil f(T_1, \dots, T_n) \rceil &= f \ \lceil T_1 \rceil \ \dots \ \lceil T_n \rceil \\
\lceil \mathbf{the} \ A \rceil &= \varepsilon \lceil A \rceil & \lceil \{T \text{ where } x_1 \text{ is } A_1, \dots, x_n \text{ is } A_n : \Phi\} \rceil &= \\
\mathbf{replSep}_n \lceil A_1 \rceil (\lambda x_1. A_2) \dots (\lambda x_1 \dots x_{n-1}. \lceil A_n \rceil) (\lambda x_1 \dots x_n. \lceil T \rceil) (\lambda x_1 \dots x_n. \lceil \Phi \rceil) \\
\lceil P(T_1, \dots, T_n) \rceil &= P \ \lceil T_1 \rceil \ \dots \ \lceil T_n \rceil & \lceil p(T_1, \dots, T_n) \rceil &= p \ \lceil T_1 \rceil \ \dots \ \lceil T_n \rceil \\
\lceil S = T \rceil &= \lceil S \rceil =_{\iota} \lceil T \rceil & \lceil S \text{ in } T \rceil &= \lceil S \rceil \in \lceil T \rceil & \lceil \mathbf{not} \ \Phi \rceil &= \neg \lceil \Phi \rceil \\
\lceil \Phi \ \& \ \Psi \rceil &= \lceil \Phi \wedge \Psi \rceil & \lceil \Phi \ \mathbf{or} \ \Psi \rceil &= \lceil \Phi \vee \Psi \rceil & \lceil \Phi \ \mathbf{implies} \ \Psi \rceil &= \lceil \Phi \rightarrow \Psi \rceil \\
\lceil \Phi \ \mathbf{iff} \ \Psi \rceil &= \lceil \Phi \leftrightarrow \Psi \rceil & \lceil \mathbf{for} \ x \ \mathbf{being} \ A \ \mathbf{holds} \ \Phi \rceil &= \forall x. \lceil A \rceil x \rightarrow \lceil \Phi \rceil \\
\lceil \mathbf{ex} \ x \ \mathbf{being} \ A \ \mathbf{st} \ \Phi \rceil &= \exists x. \lceil A \rceil x \wedge \lceil \Phi \rceil \\
& * \text{ where } x \text{ is a fresh variable of type } \iota
\end{aligned}$$

Fig. 2. Definition of the translation

mapped to THF0 compliant names in practice. In order to map Mizar schemes we define $\ulcorner (I, \Phi) \urcorner$ for M-statements by a final recursion over the prefix I :

- $\ulcorner (\cdot, \Phi) \urcorner = \ulcorner \Phi \urcorner$.
- $\ulcorner ((x : A, I), \Phi) \urcorner = \forall x. \ulcorner A \urcorner x \rightarrow \ulcorner (I, \Phi) \urcorner$.
- $\ulcorner ((F(A_1, \dots, A_n) : B, I), \Phi) \urcorner = \forall F. (\forall x_1. \ulcorner A_1 \urcorner x_1 \rightarrow \dots \rightarrow \forall x_n. \ulcorner A_n \urcorner x_n \rightarrow \ulcorner B \urcorner (F x_1 \dots x_n)) \rightarrow \ulcorner (I, \Phi) \urcorner$.
- $\ulcorner ((P[A_1, \dots, A_n], I), \Phi) \urcorner = \forall P. \ulcorner (I, \Phi) \urcorner$.

As a Mizar development is processed, new definitions are processed and the corresponding higher-order information must be declared in the problems which use this new information. We consider a few examples from early in the MML.

A simple example of a definition of an attribute is **empty** given in `xbool_e_0` [5]:

```
definition
  let X be set;
  attr X is empty means
    :Def1:
    not ex x being set st x in X;
end;
```

MPPTP creates a name `v1_xbool_e_0` of type ιo . Note that simply due to its type, `v1_xbool_e_0` can be used as an attribute and mode to form M-types. It can also be used to form M-propositions. In the Mizar development, **empty** the proposition X is **empty** corresponds to the M-proposition `v1_xbool_e_0(X)` which translates to the higher-order proposition `v1_xbool_e_0 X`. For particular problems, MPPTP also exports relevant axioms about `v1_xbool_e_0`. For example, its definition translates to $\neg \exists x. \top \wedge x \in X$ (or, equivalently, $\neg \exists x. x \in X$).

The most common example of a mode used in this paper is **Element of** from the Mizar article `subset_1` [13]:

```
definition
  let X;
  mode Element of X means :Def1:
  it in X if X is non empty otherwise it is empty;
  ...
```

Since this is the first mode definition in the article, the corresponding name created by MPPTP is `m1_subset_1`, declared to have type ιo . That is, `m1_subset_1` expects two arguments of type ι and yields a proposition. The Mizar type **Element of** X corresponds to the M-type `m1_subset_1(·, X)` which maps to the term $\lambda x_\iota. \text{m1_subset_1 } x \ X$. Note that the dependent Mizar type **Element of** X maps to a term of type ιo with a free variable X (making the dependency explicit). For the sake of readability, we will write $s \hat{\in} t$ for `m1_subset_1 s t`. Note that since Mizar requires all types to be nonempty, the **Element of** mode is defined so that $x \hat{\in} X$ if and only if either X is nonempty and $x \in X$ or both X and x are empty. That is, if X is nonempty, then $x \hat{\in} X$ if and only if $x \in X$, as expected. However, $x \hat{\in} \emptyset$ if and only if $x = \emptyset$, which may be surprising when it is first encountered.

Finally, we examine examples of schemes to see how M-statements are translated in practice.

The MML includes Fraenkel's Replacement axiom scheme as an axiom of TG. As formulated in Mizar, the scheme asserts that for each set A and each binary relation P on sets, if P is functional, then there is a set X such that $x \in X$ iff there is a $y \in A$ such that $P(y, x)$ [11]. In Mizar's syntax, the scheme is specified as follows:

```
scheme Fraenkel { A()-> set, P[set, set] }:  
  ex X st for x holds x in X iff ex y st y in A() & P[y,x]  
  provided for x,y,z st P[x,y] & P[x,z] holds y = z
```

This can be seen as an M-statement with prefix $A : \text{set}, P[\text{set}, \text{set}]$ and an M-proposition corresponding to the body. The M-statement translates to the higher-order proposition

$$\forall A_L. \forall P_{LO}. (\forall xy z_L. Pxy \wedge Pxz \rightarrow y = z) \rightarrow \exists X_L. \forall x_L. x \in X \leftrightarrow \exists y. y \in A \wedge Pyx.$$

An early application of the Fraenkel scheme is to prove Zermelo's Separation scheme discussed at the end of Sect. 3, where the corresponding M-statement is given. The M-statement translates to the following higher-order proposition:

$$\forall A_L. \forall P_{LO}. \exists X_L. \forall x_L. x \in X \leftrightarrow x \in A \wedge Px.$$

For each scheme proven in the MML, the MPTP system has generated a corresponding higher-order problem in THF0 format [9]. For example, the problem corresponding to the separation scheme is `s1_xboole.0`. In order to prove `s1_xboole.0` automatically, a prover would need to synthesize the appropriate relation to use with Replacement, e.g., $\lambda xy_L. x = y \wedge Py$ where P is the predicate from Separation. At the moment, neither Satallax nor LEO-II can prove this automatically.

The Mizar proof begins by defining a predicate Q and then applying Replacement with Q .

```
defpred Q[set,set] means $1 = $2 & P[$2];  
A1: for x,y,z st Q[x,y] & Q[x,z] holds y = z;  
  consider X such that  
A2: for x holds x in X iff ex y st y in A() & Q[y,x]  
                                     from TARSKI:sch 1(A1);
```

In λ -notation, the definition of Q is $\lambda xy_L. x = y \wedge Py$. Line A1 justifies $y = z$ whenever Qxy and Qxz . When schemes are used to justify Mizar proof steps, the keyword **from** is used. These are the steps we classify as *scheme justifications*. In this case, the Replacement scheme is used to justify the existence of a set X such that $x \in X$ iff $\exists y. y \in A \wedge Qyx$. A higher-order problem can be extracted from each such scheme justification. For this particular example, the conjecture to prove is $\exists X. \forall x. x \in X \leftrightarrow \exists y. y \in A \wedge y = x \wedge Px$. This follows from Replacement and A1, but requires instantiating the higher-order variable in the Replacement

axiom with Q . Note that Q is not explicitly given in the problem, but can easily be recovered using pattern unification [7], as we now demonstrate. Suppose we replace the outermost quantifiers in the Replacement axiom with existential variables $\mathcal{A}$ of type ι and $\mathcal{R}$ of type $\iota\iota$. The conclusion of the implication has the following form:

$$\exists X_{\iota}.\forall x_{\iota}.x \in X \leftrightarrow \exists y.y \in \mathcal{A} \wedge \mathcal{R}yx.$$

Since the subterm $\mathcal{R}yx$ is the higher-order existential variable $\mathcal{R}$ applied to distinct bound variables (y and x), we can use pattern unification (in this case pattern matching) to obtain solutions for $\mathcal{A}$ and $\mathcal{R}$. That is, when we match against

$$\exists X.\forall x.x \in X \leftrightarrow \exists y.y \in A \wedge y = x \wedge Px$$

we obtain the disagreement pairs $X, x, y | \mathcal{A} =^? A$ and $X, x, y | \mathcal{R}yx =^? y = x \wedge Px$ which has the unique (desired) solution: A for $\mathcal{A}$ and $\lambda yx.y = x \wedge Px$ for $\mathcal{R}$. Neither Satallax nor LEO-II re-prove this scheme justification within 5 min with the default strategy schedule. However, Satallax is able to prove the problem corresponding to this scheme justification under certain flag settings that encourage pattern unification.

5 Experiments

We now report on the results of running two higher-order automated theorem provers (Satallax and LEO-II) on some of the problems resulting from the translation described in the previous section. We consider four problem sets:¹

- **SimpGC**: Simple justifications where the conclusion includes a global choice operator. From four Mizar articles [1, 10, 12, 13] 47 problems were extracted.
- **SimpFr**: Simple justifications where the problem contains a Fraenkel term. We consider 245 such problems arising from three Mizar articles [1, 10, 12]. Since these proved to be surprisingly difficult, we also considered “pruned” versions of the problems in which the first-order theorem prover E [8] indicated which axioms it used to find a corresponding first-order proof.
- **SchJust**: For each scheme justifications (using `from`) in a Mizar proof in the MML, a corresponding problem was created. There are 10192 such problems.
- **SchPfs**: Out of 787 schemes proven in the MML, 610 have a proof making use of a scheme justification. For each of these 610 we have created a corresponding problem. Note that solving these problems requires finding a full proof, not justifying a single Mizar step in a proof. Hence these should be harder than the previous problem sets.

The results of running Satallax and LEO-II on the problem sets with the default settings and a time limit of 5 min are shown in Table 1. In addition, we note the

¹ The THF versions of the problems discussed here are available from <http://147.32.69.25/~chad/mptp-thf.tgz>.

number of problems both provers solved. For the remainder of the section, we discuss the results and describe some concrete examples.

One of the first uses of the global choice operator in Mizar is to define a (first-order) choice operator on sets called **choose** [13].

```

definition
  let S be set;
  func choose S -> Element of S equals
    the Element of S;
  correctness;
end;

```

Table 1. Results on problem sets with 5 min time limit

	Total problems	Satallax	LEO-II	Either
SimpGC	47	24 (51 %)	28 (60 %)	30 (64 %)
SimpFr	245	126 (52 %)	88 (36 %)	165 (67 %)
SimpFr pruned	245	159 (65 %)	155 (63 %)	192 (78 %)
SchJust	10192	5608 (55 %)	1524 (15 %)	6072 (60 %)
SchPfs	610	31 (5 %)	67 (11 %)	81 (13 %)

Note that no proof is given for correctness, as Mizar recognizes that **the Element of S** has type **Element of S**. Let us consider the corresponding higher-order simple justification problem. The higher-order problem would include the declaration of ε from Fig. 1. In addition, the fact that types of the form **Element of A** are nonempty is given: $\forall A_l. \exists B_l. B \hat{\in} A$. The conjecture to justify is

$$\varepsilon(\lambda A_l. A \hat{\in} c) \hat{\in} c$$

for a fixed c . This, of course, follows immediately from the two axioms and both Satallax and LEO-II can easily re-prove this simple justification.

Note that simply because a simple justification has a conclusion with a global choice operator does not mean that the choice axiom plays a role in the justification. Indeed, for the two examples from the problem set **SimpGC** Satallax proves but LEO-II does not, the proofs Satallax finds do not use the axiom about ε . Furthermore, upon inspection it became clear that some problems neither prover could solve also do not require the axiom about ε . Consider the following fragment of a Mizar proof about group theory [12].

```

set a = the Element of G;
...
consider b such that
A4: H * a = {b} by A1;
  h * a in H * a by A3,Th104;
  then
A5: h * a = b by A4,TARSKI:def 1;

```

The final justification is essentially the definition of singleton. The only reason the corresponding higher-order problem falls into class **SimpGC** is because a is $\varepsilon(\lambda x.x\hat{\in}(c\ G))$ (where c is a function taking a group to its carrier set, left implicit in the Mizar text). The fact that neither Satallax nor LEO-II could solve this problem was due to the fact that there are too many extra (unnecessary) axioms given in the generated problem. After pruning away the unnecessary axioms (with the help of E prover on a corresponding first-order problem), both Satallax and LEO-II can prove the pruned problem. LEO-II proves the pruned problem within 8 s and Satallax proves the pruned problem in less than a second.

We now turn to the problem set **SimpFr**: simple justifications involving at least one Fraenkel term, either in the conclusion or in one of the assumptions MPTP included in the problem. There were 640 such examples in the four Mizar articles we considered, but with experimentation it became clear that often the Fraenkel term was in an assumption that was unnecessary for the proof. In order to obtain a reasonable problem set, we used E on corresponding first-order problems to obtain pruned versions of the 640 problems. (In cases where E could not find the proof, we omitted the problem.) After pruning, there were 245 problems that still included a Fraenkel term. On each of these 245 problems, we ran Satallax and LEO-II on both the original and pruned problems. On the original versions, only 20% of the problems could be solved by both provers, whereas on the pruned versions, 50% could be solved by both provers. This suggests that better relevance filtering would be one of the most important potential improvements.

We briefly examine two small examples involving Fraenkel terms. Consider the following proof fragment from [10].

```
assume a in { x1 : x1 in A1 & not x1 in B1 or not x1 in A1 & x1 in B1 };
then ex x1 st a = x1 &
      (x1 in A1 & not x1 in B1 or not x1 in A1 & x1 in B1);
```

In the context of this fragment, x_1 ranges over elements of a nonempty set X_1 . Mizar is able to verify the correctness of the last line from the first line without any explicit references as this is simply the property of membership in a Fraenkel term. In the corresponding higher-order problem, the elimination principle replSepE_1 is required for the justification. Satallax can prove the corresponding problem in less than a second. The first mode in the default strategy schedule that finds the proof is one making use of pattern unification. In particular, after replacing the outermost quantifiers of replSepE_1 with existential variables $\mathcal{A}$, $\mathcal{F}$, $\mathcal{P}$ and $\mathcal{Y}$, the proposition has the form:

$$\mathcal{Y} \in \text{replSep}_1\ \mathcal{A}\ \mathcal{F}\ \mathcal{P} \rightarrow \exists x_l. \mathcal{A}x \wedge \mathcal{P}x \wedge \mathcal{Y} = \mathcal{F}x.$$

All the occurrences of the existential variables are pattern occurrences, and so pattern matching can be used to find the appropriate instances. In particular, one axiom of the problem is

$$a \in \text{replSep}_1\ (\lambda x_l.x\hat{\in}X)\ (\lambda x_l.x)\ (\lambda x_l.x \in A \wedge x \notin B \vee x \notin A \wedge x \in B).$$

When the antecedent of the implication above is matched against this axiom, the following instantiations result:

- $\mathcal{Y} := a$
- $\mathcal{A} := \lambda x_l. x \in X$
- $\mathcal{F} := \lambda x_l. x$
- $\mathcal{P} := \lambda x_l. x \in A \wedge x \notin B \vee x \notin A \wedge x \in B$

Given these instantiations, the solution is immediate. Satallax can prove both the pruned and unpruned version of this example in less than a second. LEO-II timed out after five minutes on both versions.

We consider a simple justification requiring the `replSepI1`. Consider the following proof fragment from [10]:

```
A2: a = x1 and
A3: P[x1];
   Q[x1] by A1,A3;
   hence thesis by A2;
```

where the thesis in the last step is

`a in { z1 where z1 is Element of X1: Q[z1] }`

As in the previous example, `x1` ranges over elements of a nonempty set `X1`. In the higher-order problem corresponding to the final simple justification (by `A2`), the conjecture has the form $a \in \text{replSep}_1 (\lambda x_l. x \in X) (\lambda x_l. x) (\lambda x_l. Qx)$. In addition `replSepI1`, the axioms needed for the proof are $x_1 \in X$ (using the type of `x1` in the Mizar article), $a = x_1$ (from `A2` in the proof fragment above), Qx_1 (from the previous step in the proof fragment above) and the extra axiom $\forall X_l. \text{sethood} (\lambda x_l. x \in X)$. Satallax requires roughly 6 s before reaching a mode in the default strategy schedule that can solve this problem. The successful mode requires less than a second to find the proof. Again, the mode makes use of pattern unification to find the proper instantiations. LEO-II can also find the proof in this example, and takes just under 6 s.

Lastly we turn to scheme justifications (**SchJust**) and full proofs of schemes (**SchPfs**). In Sect. 4 we have already discussed an example of a scheme that cannot be automatically proven (Separation from Replacement) by either prover. In addition we saw that neither prover could even re-prove the relevant scheme justification in the Mizar proof of Separation from Replacement within 5 min using the default settings.

Satallax performed significantly better than LEO-II on scheme justifications, while LEO-II performed significantly better than Satallax on proofs of full schemes. We consider one example of a scheme justification that Satallax solved but LEO-II did not. We then consider an example of a full scheme that LEO-II solved but Satallax did not.

The set operation $X \setminus Y$ is defined in an early Mizar article [5], and the following required existence proof is given:

```
defpred P[set] means not $1 in Y;
thus ex Z being set st for x holds x in Z iff x in X & P[x]
from Separation;
```

Note that the scheme justification makes use of the Separation scheme using the set X and the predicate $\lambda x.x \notin Y$. Again, the higher-order instantiation $\lambda x.x \notin Y$ can be determined using pattern matching, and Satallax can re-prove this in a fraction of a second using such a mode. With the default strategy schedule, Satallax tries such a mode and solves the problem in 37s. LEO-II times out after 5 min.

A scheme LEO-II can fully prove but Satallax cannot is the following Mizar scheme [13]:

```
scheme SubsetEx { A() -> non empty set, P[set] } :
  ex B being Subset of A() st
  for x being Element of A() holds x in B iff P[x]
```

This is again a form of Separation and is proven using the Separation scheme already considered. The primary difference between the schemes is that the new scheme `SubsetEx` asserts that the set has type `Subset of A` (notation for `Element of  $\wp A$`) and restricts the inner universal quantifier to `Element of A`. In the corresponding higher-order problem, we must prove the formula

$$\exists B. B \hat{\in} \wp A \wedge \forall x. x \hat{\in} A \rightarrow (x \in B \leftrightarrow Px)$$

from the higher-order formula

$$\forall Q. \forall X. \exists B. \forall x. x \in B \leftrightarrow x \in X \wedge Qx.$$

The solution is simple: instantiate the assumption with the $Q := P$ and $X := A$ giving an appropriate witness B for the conjecture. Some minor first-order reasoning completes the proof. LEO-II can find the proof by doing some clause normalization and calling E. It is E that does the “higher-order” instantiation of P for Q and completes the proof. This is possible since the higher-order problem, after being encoded into first-order, is still provable. (In particular, the proof does not require β -reductions.) Satallax, on the other hand, does not solve the problem and times out after 5 min. The minor structural differences between the assumption and conclusion prevents pattern matching from suggesting the instantiation P for Q . While P is among the possible instantiations considered for Q , other possible instantiations are considered as well. The combination of multiple possible instantiations and required first-order reasoning makes the problem out of reach for the current version of Satallax.

6 Conclusion

We have described an extension of MPTP that creates higher-order theorem proving problems from the MML. The resulting problems seem to present challenges for higher-order theorem provers. For example, even some of the easiest problems become difficult if there are too many axioms, so better relevance filtering is necessary. Even simple reasoning about Fraenkel terms seems to be more difficult than one would expect, and so these examples may provide insights into improvements that can be made to automated provers.

There are multiple possibilities for the translation of Fraenkel terms that bind more than one set variable. We have implemented one way and suggested another. Further experimentation will likely be helpful for determining a good way to handle these cases.

The problems generated from scheme justifications and full proofs of schemes turned out to show the different strengths and weaknesses of Satallax and LEO-II. Hopefully such problem sets will lead to improvements in higher-order automated theorem provers. Given enough improvement on such problems, perhaps higher-order automated provers could provide help to Mizar authors who make use of the features of Mizar that go beyond first-order. In order to serve this purpose, care would have to be taken that the automated provers do not search for proofs that go beyond Mizar's logic (e.g., make use of higher-order quantifiers within instantiations). We leave such concerns to future work.

References

1. Bancerek, G.: On the characteristic and weight of a topological space. *Formalized Math.* **13**(1), 163–169 (2005)
2. Benzmüller, C., Paulson, L.C., Sultana, N., Theiß, F.: The higher-order prover LEO-II. *J. Autom. Reasoning* **55**(4), 389–404 (2015)
3. Brown, C.E.: Reducing higher-order theorem proving to a sequence of SAT problems. *J. Autom. Reasoning* **51**(1), 57–77 (2013)
4. Church, A.: A formulation of the simple theory of types. *J. Symbol. Logic* **5**, 56–68 (1940)
5. Committee, L.: Boolean properties of sets – definitions, April 2002. <http://mizar.org/JFM/EMM/xboole.0.html>
6. Grabowski, A., Kornilowicz, A., Naumowicz, A.: Mizar in a nutshell. *J. Formalized Reasoning* **3**(2), 153–245 (2010)
7. Miller, D., Nadathur, G.: A logic programming approach to manipulating formulas and programs. In: *IEEE Symposium on Logic Programming*. Salt Lake City (1987)
8. Schulz, S.: E - A Brainiac theorem prover. *J. AI Commun.* **15**(2/3), 111–126 (2002)
9. Sutcliffe, G., Benzmüller, C.: Automated reasoning in higher-order logic using the TPTP THF infrastructure. *J. Formalized Reasoning* **3**(1), 1–27 (2010)
10. Trybulec, A.: Domains and their Cartesian products. *Formalized Math.* **1**(1), 115–122 (1990)
11. Trybulec, A.: Tarski Grothendieck set theory. *Formalized Math.* **1**(1), 9–11 (1990)
12. Trybulec, W.A.: Subgroup and cosets of subgroups. *Formalized Math.* **1**(5), 855–864 (1990)

13. Trybulec, Z.: Properties of subsets. *Formalized Math.* **1**(1), 67–71 (1990)
14. Urban, J.: MPTP 0.2: design, implementation, and initial experiments. *J. Autom. Reasoning* **37**(1–2), 21–43 (2006)
15. Urban, J.: Translating Mizar for first order theorem provers. In: Asperti, A., Buchberger, B., Davenport, J.H. (eds.) *MKM 2003*. LNCS, vol. 2594, pp. 203–215. Springer, Heidelberg (2003). doi:[10.1007/3-540-36469-2_16](https://doi.org/10.1007/3-540-36469-2_16)
16. Urban, J.: MPTP - motivation, implementation, first experiments. *J. Autom. Reasoning* **33**(3), 319–339 (2005). <http://dx.org/10.1007/s10817-004-6245-1>